Week 7 Lab: Employee Management System (Lists and Data Management)

Introduction

This is Phase 2 of this 4-phase course project. In this lab you modify your Phase 1 program to store employee data in lists and calculate statistics, enhance your program to remember all employee records (not just totals), and generate statistical reports.

What you'll add in Phase 2:

- Eight parallel lists to store all employee data
- A detailed display showing all employees processed
- Statistical calculations using built-in functions (average, highest, lowest)
- Functions that work with multiple list parameters
- Tuple unpacking to receive multiple return values

Learning Objectives

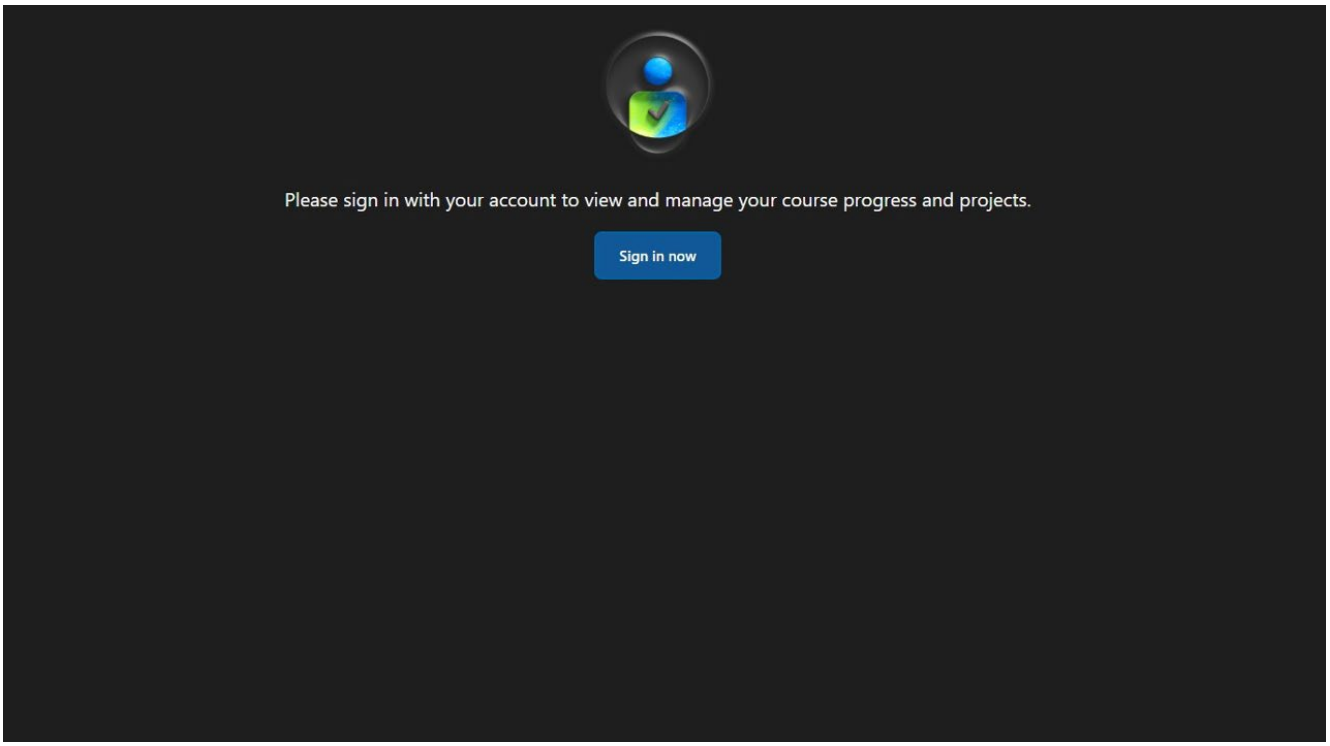
By completing this phase, you will demonstrate your ability to:

- Create and initialize multiple lists
- Use the **.append()** method to build lists dynamically
- Iterate through lists using **for** loops with **range(len())**
- Keep multiple parallel lists synchronized
- Pass multiple list parameters to functions
- Use Python's built-in functions **(sum, len, max, min**)
- Return multiple values from a function
- Use tuple unpacking to receive multiple return values
- Display data from coordinated lists

Lab Instructions

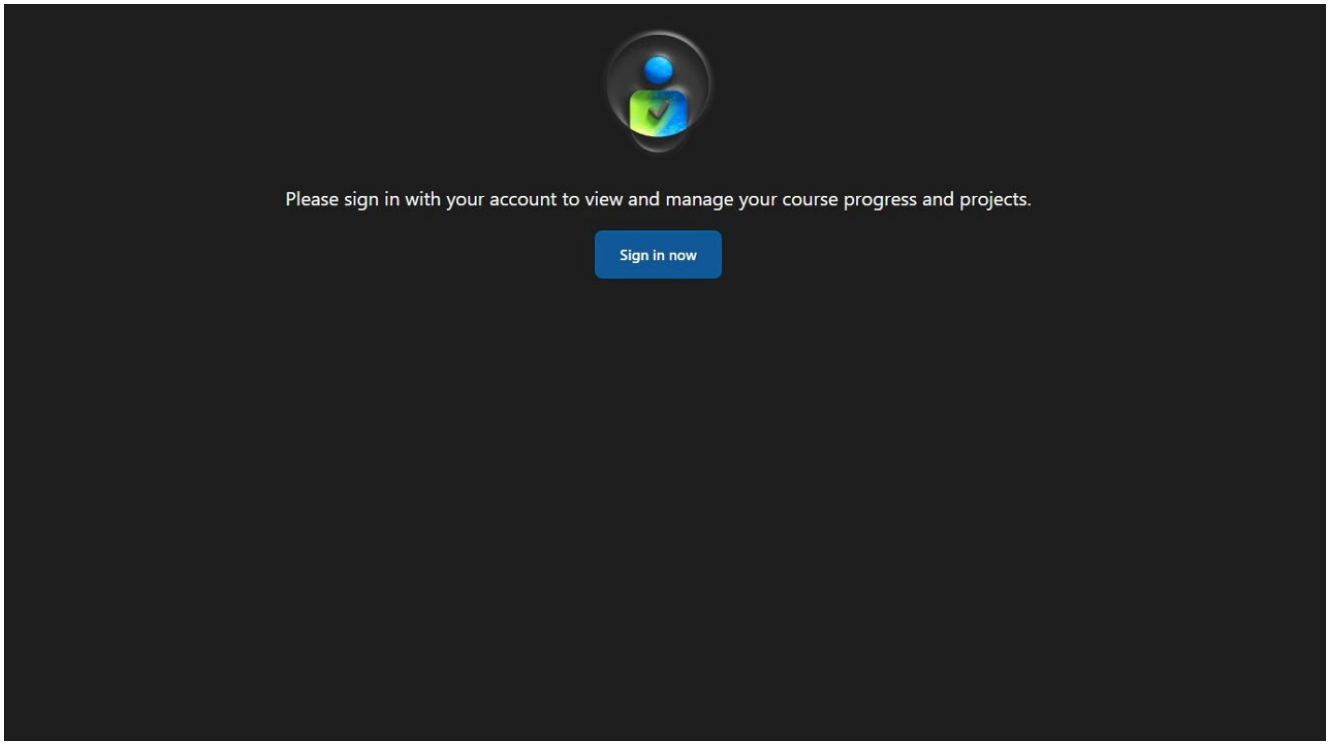
Step 1

Click on https://vscodeedu.com/my-work/projects to access Visual Studio Code for Education.

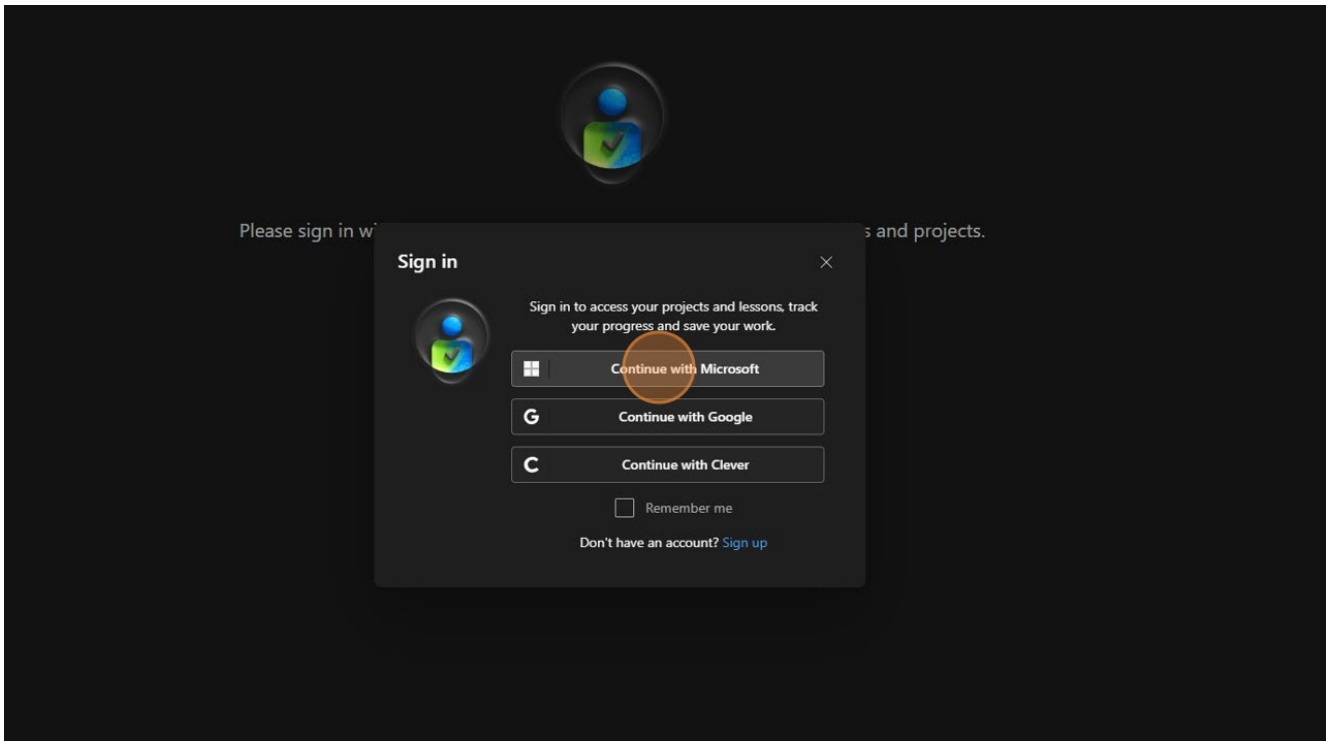


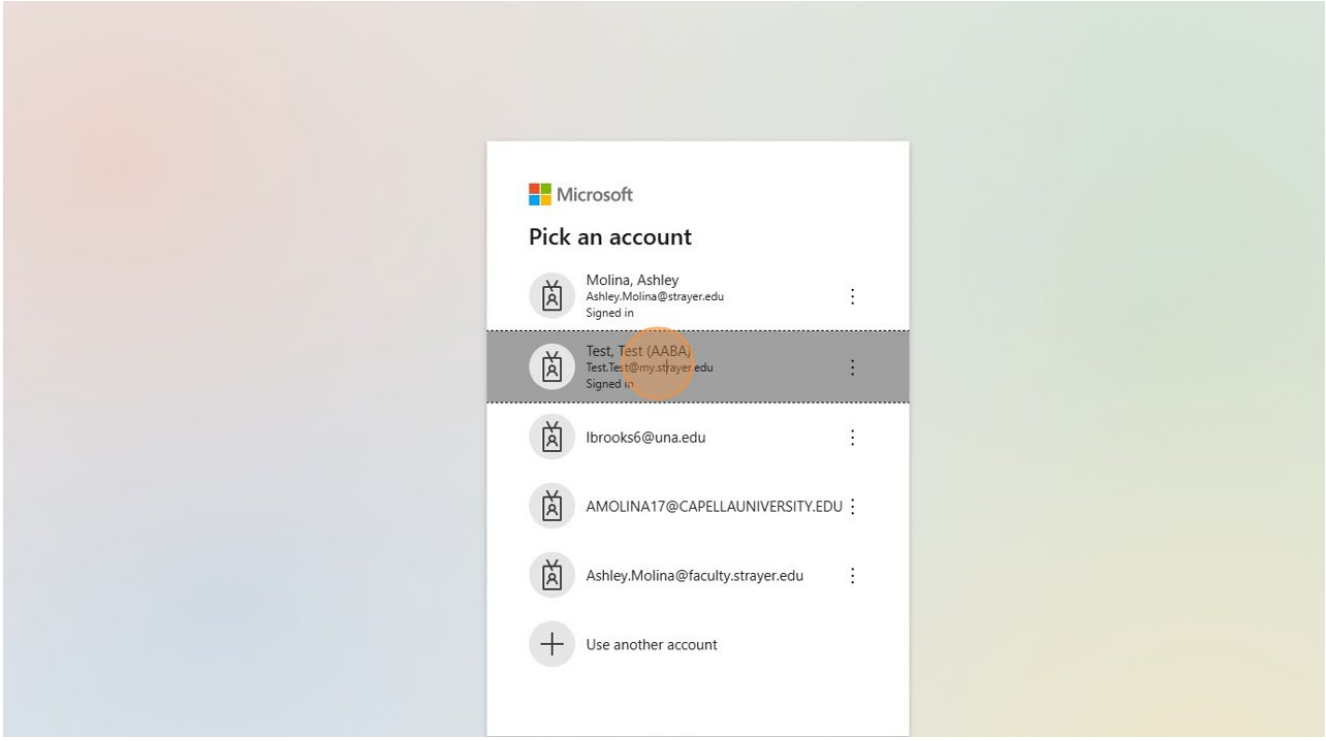
Step 2

You will then Click ‘Sign in now’.

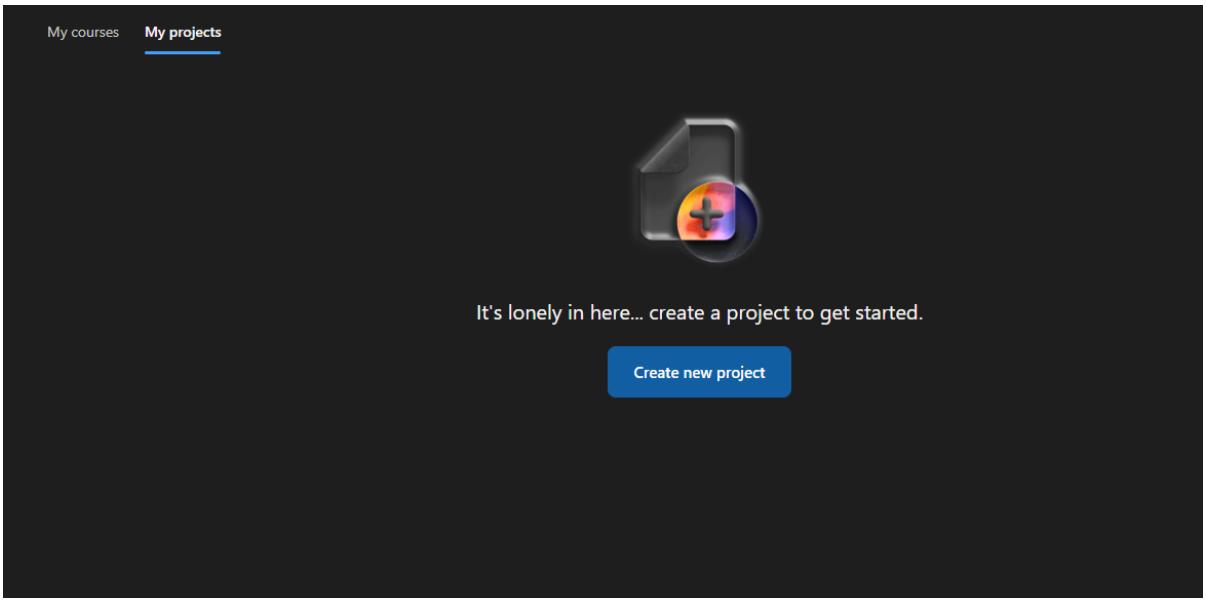


Then select **‘Continue with Microsoft’ or ‘Continue with Google’** to sign in with your already established email address. Using ‘Continue with Microsoft’ you will sign in with your Strayer email address that ends in @my.strayer.edu and your iCampus password. You will be prompted to log into the Strayer Systems as normal.


Please sign in w
s and projects.
Sign in
Sign in to access your projects and lessons, track your progress and save your work.
Continue with Microsoft
Continue with Google
Continue with Clever
Remember me
Don't have an account? Sign up

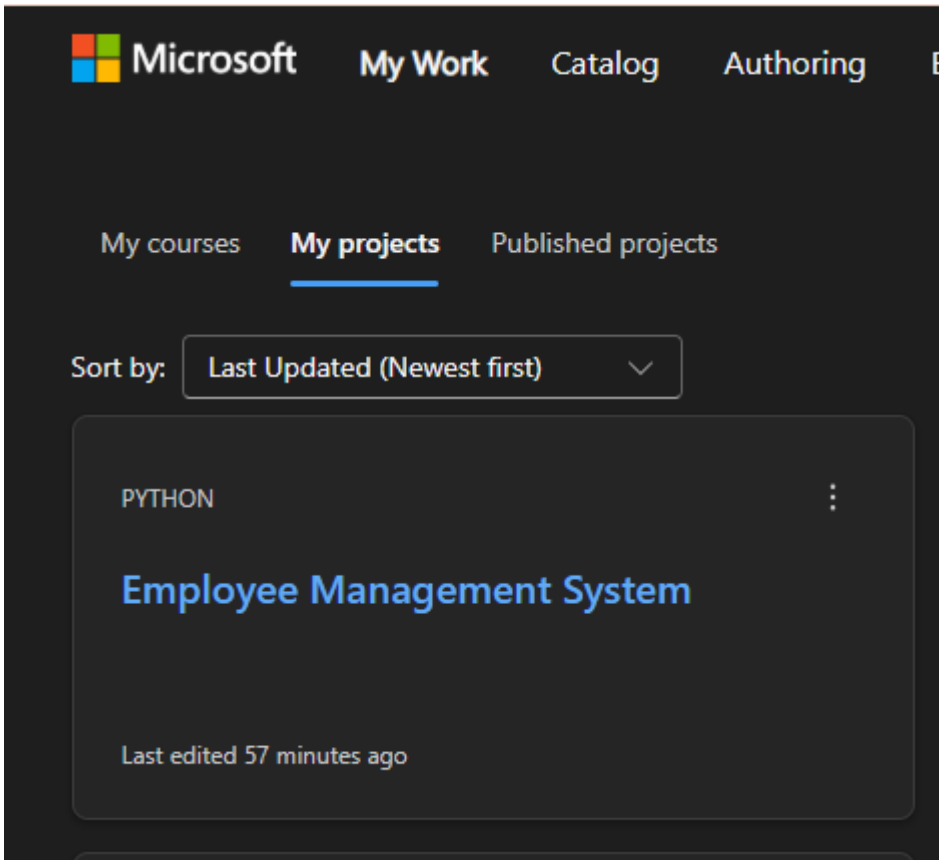

Microsoft
Pick an account
Molina, Ashley
Ashley.Molina@strayer.edu
Signed in
Test, Test (AABA)
Test.Test@my.strayer.edu
Signed in
lbrooks6@una.edu
AMOLINA17@CAPELLAUNIVERSITY.EDU
Ashley.Molina@faculty.strayer.edu
Use another account

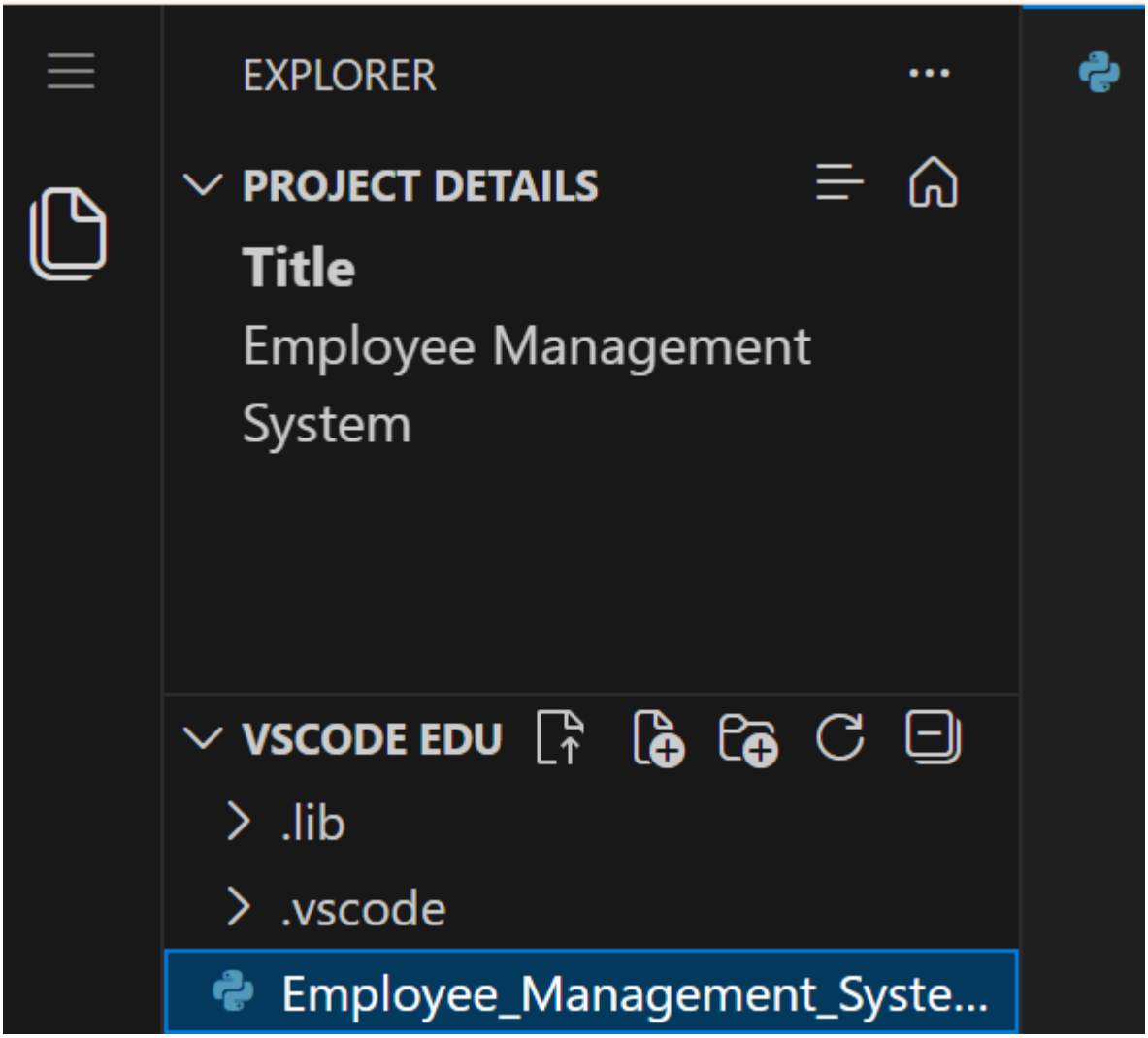
Step 3



Step 4

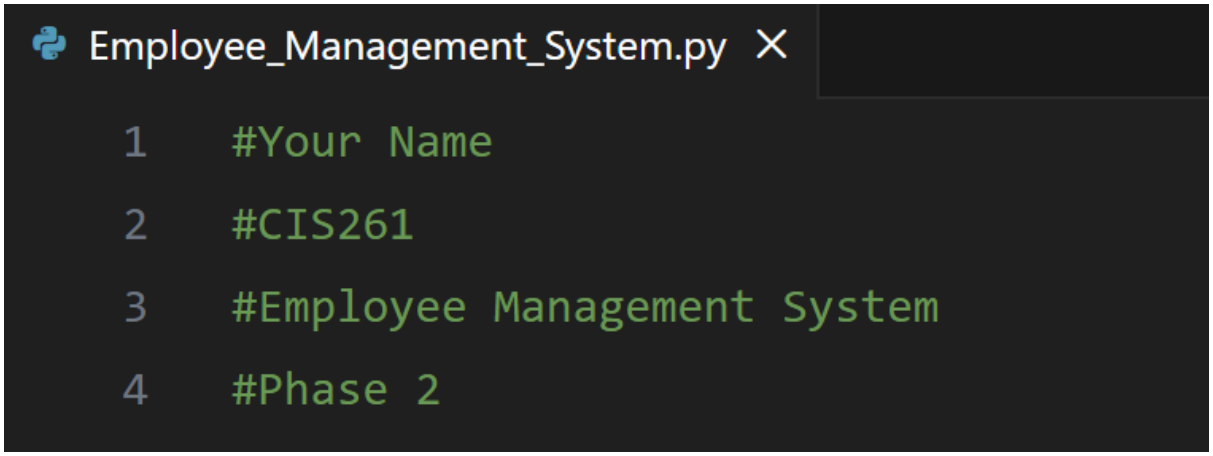
Open your previous project named **Employee Management System** and click on the file name Employee_Management_System.py from the left-hand menu once it loads to bring up your previous code.





Step 5

Edit your line 4 comment to #Phase 2.



Step 6

Write a program to perform the following functionality.

Part 1
Create eight empty lists to store employee data. Initialize these lists in your main() function before the loop:

- **first_names** - List to store first names
- **last_names** - List to store last names
- **employee_ids** - List to store employee IDs
- **hours_list** - List to store hours worked
- **rates_list** - List to store hourly rates
- **gross_pays** - List to store gross pay amounts
- **deductions_list** - List to store deduction amounts
- **net_pays** - List to store net pay amounts

Part 2
Modify your main loop to append data to all eight lists after displaying each employee record. Use the **.append()** method to add each piece of data to the corresponding list.

NOTE: All eight lists must stay synchronized - each list should have the same number of items, and the data at index 0 in each list should represent the first employee, index 1 the second employee, etc.

Part 3
Create a new function **display_all_employees()** that implements the following functionality:

- Accept eight list parameters: **first_names, last_names, employee_ids, hours_list, rates_list, gross_pays, deductions_list, net_pays**
- Use a **for** loop with **range(len())** to iterate through the lists
- Display detailed information for each employee with numbering (Employee Record #1, #2, etc.)
- Access all eight lists using the same index **[i]** inside the loop

Part 4
Create a new function **calculate_payroll_statistics()** that implements the following functionality:

- Accept one parameter: **gross_pays** (the list of gross pay amounts)
- Use built-in functions to calculate four values:
    - Total gross pay using **sum(gross_pays)**
    - Average gross pay using **sum(gross_pays) / len(gross_pays)**
    - Highest gross pay using **max(gross_pays)**
    - Lowest gross pay using **min(gross_pays)**
- Return all four values in a single return statement: **return total_gross, average_gross, highest_gross, lowest_gross**

Part 5
Modify your **display_summary()** function to accept seven parameters instead of four:

- **total_employees** - Total number of employees
- **total_gross** - Total gross pay
- **total_deductions** - Total deductions
- **total_net** - Total net pay
- **average_gross** - Average gross pay (NEW)

- **highest_gross** - Highest gross pay (NEW)
- **lowest_gross** - Lowest gross pay (NEW)

Update the function to display all seven values with appropriate labels.

Step 6
Modify your **main()** function to call the new functions after the loop ends:

- Call **display_all_employees()** with all eight lists as arguments
- Call **calculate_payroll_statistics()** and use tuple unpacking to receive the four return values: **total_gross_calc, average_gross, highest_gross, lowest_gross = calculate_payroll_statistics(gross_pays)**
- Call the modified **display_summary()** with all seven arguments

NOTE: Be sure to display the all employees list and the summary only if at least one employee was processed.

Step 7
Write and test the code for the lab.

Step 8

Testing Requirements

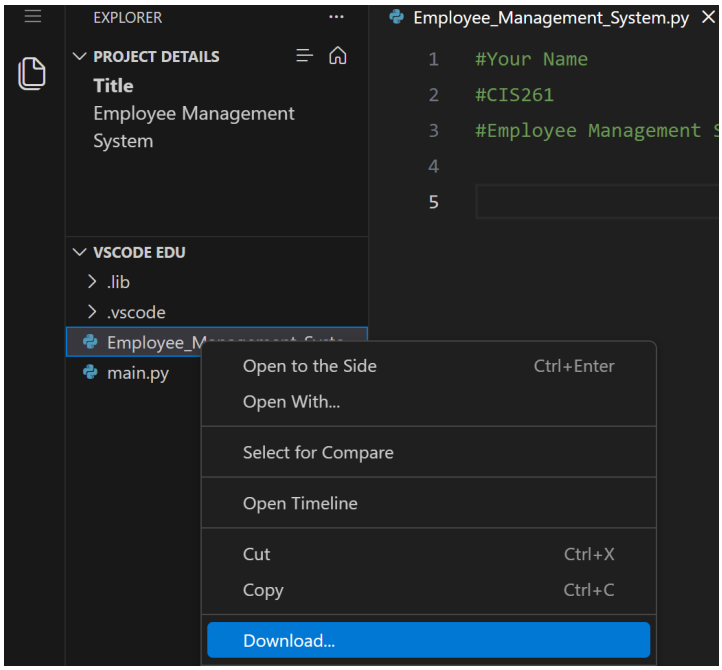
Test your program with a minimum of **3 employees** to demonstrate:

- All data correctly stored in lists (verify by checking the detailed employee list)
- For loop with **range(len())** iterates correctly
- Statistics calculated using built-in functions
- Tuple unpacking works correctly for the four return values
- All seven parameters displayed in summary
- Lists remain synchronized throughout execution

Saving Your Work to Your Computer

Now, we are going to download our python file for submission!

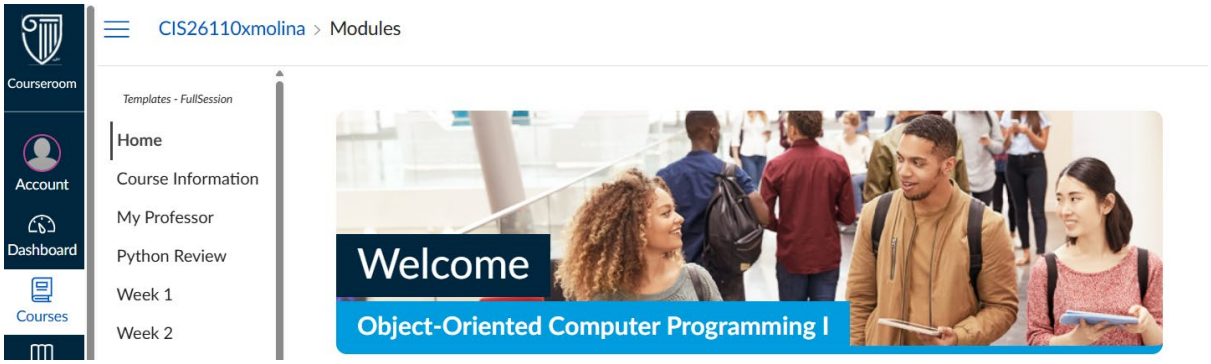
We need to download your **Employee_Management_System.py** file to your computer to prepare your submission. To do this right-click on **Employee_Management_System.py** and a pop-up box will appear. Select **Download** from the menu, to download the file to your downloads folder on your computer.



*If this is the first time in here you may get a pop-up that asks '**vscode.dev wants to See text and images copied to the clipboard Block or Allow**' click **Allow**.* Once you 'Allow' you will then see your menu to download your file.

Now your python file for this project is in your computer's downloads folder!

Go to https://canvas.strayer.edu/ in your web browser, sign-in, and select **CIS261 Object-Oriented Programming** from your tiles.



Step 3

Select Week 7 – Lab from the Week 7 menu options

For this lab you should submit your **CIS261_Employee_Management_System.py** directly to the **Week 7 – Lab** submission area.

Step 4

Select the **Upload** option and then select **Choose a file to upload** from the submission area and select **CIS261_Employee_Management_System.py** from your selected location in your File Explorer.

When you select the file you will click **Open** to add the file to Canvas.

Once your file shows a green check mark beside it click "***I agree to the tool's End-User License Agreement. This assignment submission is my own, original work"*** and click "**Submit Assignment"**

Review

Well, done, you have completed the **First Step of the Employee Management System!**

Important Notes

- You must create exactly **eight lists** with the specified names
- Use **.append()** to add data to lists (do NOT use other methods like .insert() or direct assignment)
- Must use for **i in range(len(list_name)):** to iterate through lists
- Must use built-in functions: **sum(), len(), max(), min() in calculate_payroll_statistics()**
- Must return four values from **calculate_payroll_statistics()** in a single return statement
- Must use tuple unpacking to receive the four values: a, b, c, d = function()
- All eight lists must remain synchronized (same number of items, data at same indices matches)
- Display the detailed employee list BEFORE displaying the summary

END OF DOCUMENT